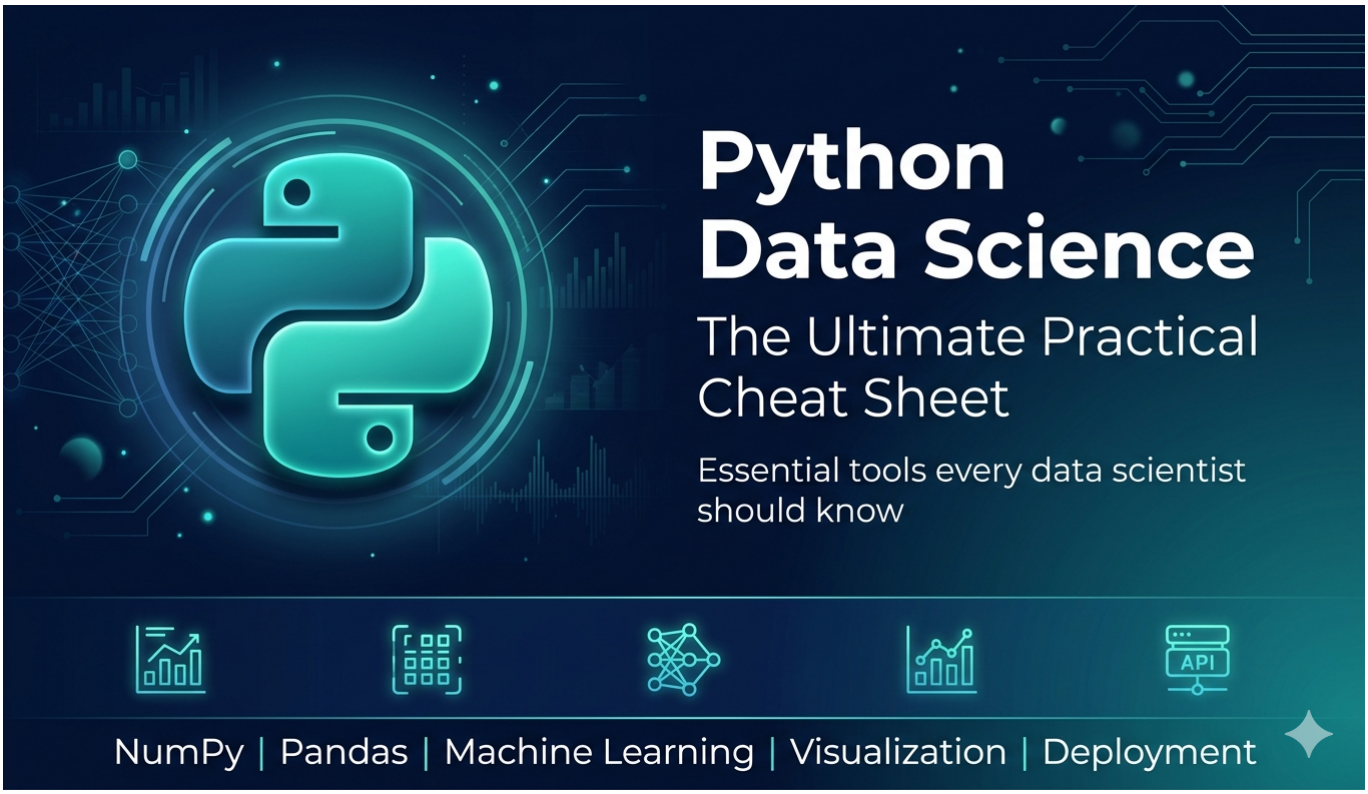
Python for Data Science

The Practical Cheat Sheet

Core tools every data scientist should know



NumPy | Pandas | Machine Learning | Deployment

Written By: *Tanu Nanda Prabhu*

Python for Data Science Cheat Sheet

A concise reference covering essential Python tools used in modern data science workflows.

This guide focuses on practical commands and common patterns used in real projects.

1. Python Basics for Data Science

Python is widely used in data science because of its readability and powerful ecosystem.

Common Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

## Output

Libraries are successfully loaded into the environment and available for use.

---

## Basic Data Structures

```python
data = [10, 20, 30]

person = {"name": "Alice", "age": 25}

coordinates = (10, 20)

print(data)
print(person)
print(coordinates)
```
```

Output

```
[10, 20, 30]
{'name': 'Alice', 'age': 25}
(10, 20)
```

Lambda Functions

```
square = lambda x: x ** 2
print(square(5))
```

Output

```
25
```

---

2. NumPy Essentials

NumPy provides efficient numerical operations and multi dimensional arrays.

Creating Arrays

```
import numpy as np

a = np.array([1,2,3])
b = np.zeros((3,3))
c = np.ones((2,4))
d = np.arange(0,10)
e = np.linspace(0,1,5)

print(a)
print(b)
print(c)
print(d)
print(e)
```

Output

```
[1 2 3]

[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

[0 1 2 3 4 5 6 7 8 9]
```

```
[0.   0.25 0.5  0.75 1.  ]
```

Array Operations

```
a = np.array([1,2,3])

print(a + 10)
print(a * 2)
print(np.sqrt(a))
print(np.mean(a))
print(np.std(a))
```

Output

```
[11 12 13]
[2 4 6]
[1.         1.41421356 1.73205081]
2.0
0.81649658
```

Matrix Operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print(np.dot(A,B))
print(np.transpose(A))
print(np.linalg.inv(A))
```

Output

```
[[19 22]
 [43 50]]

[[1 3]
 [2 4]]

[[-2.   1. ]
 [ 1.5 -0.5]]
```

---

3. Pandas Data Manipulation

Pandas is used for working with structured datasets.

Creating a DataFrame

```
import pandas as pd

data = {
    "name": ["Alice","Bob","Charlie"],
    "age": [25,30,35],
    "salary": [50000,60000,70000]
}

df = pd.DataFrame(data)

print(df)
```

Output

```
      name  age  salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   35   70000
```

Inspecting Data

```
print(df.head())
print(df.shape)
print(df.describe())
```

Output

```
      name  age  salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   35   70000

(3, 3)

             age        salary
count   3.000000      3.000000
```

```
mean   30.000000  60000.000000
std     5.000000  10000.000000
min    25.000000  50000.000000
max    35.000000  70000.000000
```

Selecting Data

```
print(df["age"])
print(df[["name","salary"]])
```

Output

```
0    25
1    30
2    35

      name  salary
0    Alice   50000
1      Bob   60000
2  Charlie   70000
```

Filtering Data

```
print(df[df["age"] > 30])
```

Output

```
      name  age  salary
2  Charlie   35   70000
```

GroupBy Example

```
grouped = df.groupby("age").mean(numeric_only=True)
print(grouped)
```

Output

```
grouped = df.groupby("age").mean(numeric_only=True)
print(grouped)
```

---

4. Data Visualization

Visualization helps identify trends and patterns.

Line Plot

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,25,30]

plt.plot(x,y)
plt.title("Line Chart")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Output

A line chart displaying values increasing from 10 to 30 across the x axis.

Seaborn Histogram

```
import seaborn as sns

sns.histplot(df["age"])
plt.show()
```

Output

A histogram showing the distribution of ages in the dataset.

---

5. Machine Learning with Scikit Learn

Scikit Learn provides many machine learning algorithms.

Train Test Split and Model Training

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[["age"]]
y = df["salary"]

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)

model = LinearRegression()
model.fit(X_train,y_train)

predictions = model.predict(X_test)

print(predictions)
```

Output

Example predicted values

```
[65000.]
```

---

6. Data Cleaning Techniques

Clean data improves model performance.

Removing Duplicates

```
df = df.drop_duplicates()
print(df)
```

Output

Duplicate rows are removed from the dataset.

Renaming Columns

```
df = df.rename(columns={"salary":"income"})
print(df.columns)
```

Output

```
Index(['name', 'age', 'income'], dtype='object')
```

---

7. Feature Engineering

Feature engineering improves model performance.

One Hot Encoding

```
cities = pd.DataFrame({"city":["NY","LA","NY"]})

encoded = pd.get_dummies(cities["city"])
print(encoded)
```

Output

```
   LA   NY
0  0    1
1  1    0
2  0    1
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaled = scaler.fit_transform(df[["age"]])

print(scaled)
```

Output

```
[[-1.2247]
 [ 0.0000]
 [ 1.2247]]
```

---

8. Model Evaluation

Model evaluation measures performance.

Regression Metrics

```
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test,predictions)
r2 = r2_score(y_test,predictions)

print(mse)
print(r2)
```

Output

Example Output

```
25000000.0
0.85
```

---

9. Model Deployment

Models can be deployed using web APIs.

FastAPI Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Model API running"}
```

Output

API response when accessing the root endpoint

```
{
 "message": "Model API running"
```

```
}
```

---

10. Performance Optimization Tips

Large datasets require efficient computation.

Vectorized Operations

```
arr = np.array([1,2,3,4])
result = arr * 2

print(result)
```

Output

```
[2 4 6 8]
```

Efficient Data Types

```
df["age"] = df["age"].astype("int32")

print(df.dtypes)
```

Output

```
name      object
age        int32
income     int64
```

Summary

Python provides a powerful ecosystem for data science.

Key components include:

- NumPy for numerical computing
- Pandas for data manipulation

- Matplotlib and Seaborn for visualization
- Scikit Learn for machine learning
- FastAPI and Flask for deployment

Mastering these tools enables efficient data analysis, model development, and production deployment.